

Сам себе DevOps

В этом задании надо будет своими руками заняться запуском нескольких контейнеров с помощью Docker Compose (и не только).

В рамках задания содержимое контейнеров не очень важно, но необходимо и играет важную роль.

Что потребуется в задании

- Установленные **Docker** и **Docker Compose**
 - Git и GitHub (даже для ценителей GitLab)
 - Базовые знания командной строки
 - Программа с простейшим приложением на любом языке программирования
-

Основное задание

Базовый набор контейнеров

В базовом случае обойдёмся без оригинальностей.

Необходимо сделать приложение из двух частей:

1. Само приложение:
 1. Принимает HTTP-запросы на какой-либо порт.
 2. На запрос `/ping` :
 1. Отвечает `pong` .
 2. Неявно сохраняет `ip`, с которого пришёл запрос.
 3. Например, если приложение слушает 80й порт и запущенно в хостовой ОС, на команду `curl localhost/ping`, мы должны увидеть `pong` в консоли.
 3. На запрос `/visits` :
 1. Отвечает числом "посещений" запроса `/ping`
 2. Например, после двух выполнений `/ping`, на команду `curl localhost/visits`, мы должны увидеть в консоли `2`.
4. Примеры фреймворков (рекомендуется остановиться на питоне):
 1. [FastAPI](#) (python)
 2. [Flask](#) (python)
 3. [httprouter](#)(golang)

4. [Gin](#) (golang)
5. [restbed](#) (C++)

2. База данных:

1. Требуется просто запустить СУБД в отдельном контейнере.
2. Любым способом создать БД и таблицу, в которой можно было бы сохранить информацию о запросах.
3. БД должна быть доступна для взаимодействия из приложения.
4. По умолчанию, предлагаю использовать [PostgreSQL](#).

Автоматизация

На самом деле, задание начинается здесь.

Но здесь всё тоже относительно просто, нужно написать:

1. Dockerfile, который собирает приложение
2. docker-compose.yml, который собирает обе части воедино: собирает приложение, предоставляет все переменные и запускает два контейнера.
 1. Для приложения должен быть проброшен порт.
 2. Порт СУБД пробрасывать не нужно.
 3. СУБД Должна сохранять данные от запуска к запуску, если данные не стёрты явным образом.

Как сдать задание

Раз уж мы тут занялись автоматизацией, проверка тоже будет автоматической (ну почти). Для выполнения задания требуется:

1. Сделать Fork [репозитория](#) на GitHub с заданием.
2. Залить решение задания в **свою копию репозитория**.
3. Сделать Pull Request **в исходном репозитории** на добавление изменений, в котором:
 1. В названии необходимо оставить своё ФИО.
 2. В описании необходимо перечислить выполненные дополнительные задания (если такие есть).
4. После создания Pull Request, запустится автоматическая проверка (через github Actions) - необходимо убедиться, что она успешно отрабатывает.
5. Если все проверки пройдены, автоматическая часть проверки закончена: здесь придётся подождать Review. При возникновении вопросов, буду писать их непосредственно в PR (настройте уведомления, чтобы не пропустить).

6. Вы великолепны, просто дождитесь обновления [таблицы с результатами](#). PR в этом случае будет закрыт, но не удалён: обсуждение можно будет продолжить.
-

Дополнительные задания

Специально оставляю достаточно широкие формулировки, чтобы был простор для творчества.

1. Скрипт инициализации БД.

1. Инициализацию БД (в базовом случае - создание таблицы) необходимо отделить явным образом в отдельный скрипт.
2. В `docker-compose.yml` необходимо добавить сервис для запуска этого скрипта.
3. HTTP-приложение должно запускаться только после того, как отработал скрипт инициализации (должно быть достигнуто средствами `docker compose`, а не ожиданием в приложении)
4. Необходимо модифицировать файл `.github/workflows/test.yml`, чтобы в нём появилась проверка успешного завершения инициализирующего скрипта.

2. Кэширование.

1. Помимо "долгоживущих" БД в веб-приложениях частенько используют дополнительные сервисы, для ускорения ответов пользователю - добавьте использование такого сервиса в своё приложение.
2. Сохраняйте в кэш актуальное число обработанных запросов.
3. Кэш должен быть отдельным сервисом.
4. Важно соблюдать консистентность: запрос с кэшем и без него должны выдавать одинаковые результаты.
5. Необходимо модифицировать файл `.github/workflows/test.yml`, чтобы в нём появилась проверка успешного запуска кеширующего сервиса.

3. Тестирование.

1. Хотя у нас и простое приложение, всегда есть шансы ошибиться. Необходимо добавить тесты, которые проверяли бы наше приложение.
2. У тестов есть важная характеристика: Покрытие (Coverage) - нужно, чтобы разработанные тесты покрывали хотя бы 60% кода (даже если кода совсем мало).
3. Необходимо модифицировать файл `.github/workflows/test.yml`, чтобы в нём появилась проверка успешного выполнения тестов (В том числе, покрытие).
4. Здесь всё зависит от выбранного языка программирования: не переусложняйте себе задачу.

4. Разные окружения.

1. В ходе разработки может возникнуть необходимость отделять окружения: удобное для разработчиков, от окружения, которое не страшно опубликовать или отдать заказчику. Для этого могут можно использовать запуск docker compose с несколькими файлами.
 2. Сделайте разделите docker-compose на разные файлы, чтобы получить два окружения:
 1. Dev: запускается только приложение в режиме, при котором на запрос `/visits` всегда отвечает -1.
 2. Prod: всё работает в полноценном режиме
 3. Поправьте `.github/workflows/test.yml`, чтобы корректно отработывала проверка.
 5. Импровизация.
 1. Предложите своё содержимое контейнеров.
 2. Должно быть минимум 2 взаимодействующих контейнера.
 3. Ограничений сверху на число контейнеров нет.
 4. Нужно актуализировать `.github/workflows/test.yml`.
 6. Я лучше знаю.
 1. Предложите обоснованное улучшение проверяющего скрипта.
 2. В этом случае сделайте отдельный PR: в случае успеха он будет влит в общий репозиторий.
 3. В описании PR опишите аргументацию: что добавляет/улучшает проверка.
 4. Каждый человек может выполнить это задание лишь однажды, даже переписав весь workflow проверки (т.е. улучшения можно предлагать, но дополнительных баллов за это не будет начислено).
 5. Если будут одинаковые предложения, балл начисляется в порядке первинства.
-

Оценки

Основное задание относительно простое, поэтому оно приносит неполный балл. К тому же, у всех разный опыт, поэтому оценки следующие:

- Бакалаврам: 0.8 балла.
- Магистрам: 0.6 балла.

Разобравшись с основным заданием, должно стать проще, поэтому дополнительные задания всем дают по 0.2 балла.

Для баланса, максимальный балл у бакалавров и магистров одинаковый = 1.4 (даже если выполните все задания).

Презумпция виновности

В угоду удобства и специфики задания, используется публичный репозиторий. В связи с этим, вы будете иметь возможность видеть решения друг друга.

С другой стороны, современные нейронки предлагают решение лучше соседей, а мне важно, чтобы заинтересованные разобрались.

Поэтому просто оставляю предупреждение: **в случае подозрений на списывание, буду ставить 0.1 балла с блокировкой дальнейшей сдачи.**

Это, в частности, означает: при использовании публичных источников, потрудитесь добавить оригинальности своему решению.

Дедлайн

В этот раз поверю в силу автоматизации и сделаю один дедлайн: последний push в PR должен быть сделан не позднее **17:02 14.11.2025**.

Если вы ну очень уж хотели что-то доделать и запушили какие-то дополнения после дедлайна, сделайте это отдельным коммитом: тогда будут шансы, что будет засчитана хоть часть задания.